Министерство образования Республики Беларусь

Учреждение образования
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

Факультет Факультет повышения квалификации и переподготовки

Кафедра Кафедра цифровых компетенций

ОТЧЁТ
по лабораторной работе 2
по дисциплине
«Моделирование бизнес-процессов»

на тему
МОДЕЛИРОВАНИЕ БИЗНЕС-ПРОЦЕССОВ. НОТАЦИЯ BPMN

Выполнил: Хаджинова К.А.

Проверил: Русак Т.В.

Минск 2026

ЦЕЛЬ РАБОТЫ

Освоить нотацию BPMN и построить модель бизнес-процесса сервиса DSLKit с пулами, дорожками, событиями, шлюзами, артефактами и потоками сообщений.

ПОСТАНОВКА ЗАДАЧИ

Индивидуальное задание требует выбрать из декомпозиции процессов предметной области один процесс и выполнить его моделирование в нотации BPMN. Предметная область выбрана в первой лабораторной работе: облачный сервис DSLKit для создания диаграмм как кода.

Для моделирования выбран процесс «Написание и проверка DSL-кода», который в функциональной модели IDEF0 обозначен как узел A2. В нём участвуют пользователь и система, а результатом служит проверенный код, готовый к построению диаграммы.

Для выполнения задания необходимо решить следующие подзадачи:

– определить границы процесса, его участников и полезный результат;

– описать успешный путь выполнения процесса;

– добавить условия, альтернативные потоки и неуспешные завершения;

– добавить артефакты: документы и хранилища данных;

– детализировать свёрнутый подпроцесс на отдельной диаграмме;

– показать взаимодействие с внешними участниками потоками сообщений.

ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ

Нотация BPMN (Business Process Model and Notation) описывает бизнес-процесс как алгоритм: на диаграмме показаны события, исполнители, документы и потоки, сопровождающие выполнение. Нотация применяется для подробного описания логики процесса нижнего уровня и для запуска схемы на исполнение в специализированных системах. Лекция по описанию процессов рекомендует выбирать BPMN, когда процессы описываются для последующей автоматизации, а схемы должны быть кроссфункциональными и понятными без специального обучения.

Различают три уровня моделирования. Описательное моделирование показывает успешный путь процесса и использует минимум элементов. Аналитическое моделирование показывает все варианты выполнения, включая ветвления и альтернативы. Исполняемое моделирование предназначено для запуска в BPMS-системе и допускает полный алфавит нотации.

Основные категории элементов: элементы потока (события, задачи и подпроцессы, шлюзы), данные (объекты данных и базы данных), соединяющие элементы (потоки управления, потоки сообщений, ассоциации), зоны ответственности (пулы и дорожки) и артефакты (сноски). Задача не раскладывается дальше, подпроцесс детализируется на отдельной диаграмме и отмечается знаком «плюс».

События бывают стартовыми, промежуточными и конечными. По триггеру различают сообщение, таймер, условие, сигнал, ошибку и другие виды. Граничное событие присоединяется к границе задачи и при срабатывании может прервать её выполнение. Шлюзы управляют потоком: параллельный шлюз (AND) разделяет и синхронизирует ветки, эксклюзивный шлюз (XOR) выбирает один из альтернативных потоков.

Пул изображает участника процесса. Внешний участник, действия которого не рассматриваются, показывается свёрнутым пулом как «чёрный ящик». Внутри пула действия распределяются по дорожкам, то есть ролям или подразделениям. Внутри пула переходы показывает поток управления (сплошная линия), между пулами взаимодействие выполняет поток сообщений (пунктирная линия), который не присоединяется к шлюзам.

Для процессов, которые читают люди, рекомендуется ограничивать алфавит нотации: использовать только свёрнутые подпроцессы, шлюзы XOR и AND и события типов простое, таймер, сообщение и останов. Задачи называют в стиле «глагол плюс существительное», события называют свершившимся фактом в прошедшем времени, выходы из XOR подписывают «Да» и «Нет», успешное и неуспешное завершение показывают разными конечными событиями. Диаграмму строят по семи шагам: границы, успешный путь, условия и альтернативы, неуспешные завершения, артефакты, раскрытие подпроцессов, потоки сообщений к внешним пулам.

1 ХОД РАБОТЫ

1.1 Выбор процесса

Из декомпозиции процессов сервиса DSLKit, построенной в первой лабораторной работе, выбран процесс «Написание и проверка DSL-кода» (узел A2). В нём участвуют пользователь и система, есть логика проверки с альтернативами и передача результата внешнему сервису. Диаграммы построены в редакторе draw.io по правилам нотации BPMN и по рекомендациям методички для диаграмм, которые читают люди.

1.2 Определение границ процесса

Первый шаг алгоритма: определены стартовое и конечные события, участники и полезный результат. Процесс начинается, когда пользователь открыл редактор. Процесс показан открытым пулом, который назван по процессу: «Написание и проверка DSL-кода». Внутри пула две дорожки, названные по ролям: «Пользователь» и «Система DSLKit». Сервис рендеринга Kroki, которому передаётся результат, показан свёрнутым пулом как внешний участник, действия которого не рассматриваются. Полезный результат процесса – проверенный DSL-код, сохранённый в проекте и готовый к построению диаграммы.

1.3 Успешный путь

Второй шаг: описан успешный путь. Пользователь выбирает нотацию, система создаёт заготовку кода, пользователь пишет DSL-код и отправляет его на проверку. Проверка выполняется в свёрнутом подпроцессе. Если код допустим, система сохраняет его в проекте, и процесс завершается событием с сообщением «Код передан на построение». Основная диаграмма показана на рисунке 1.1.

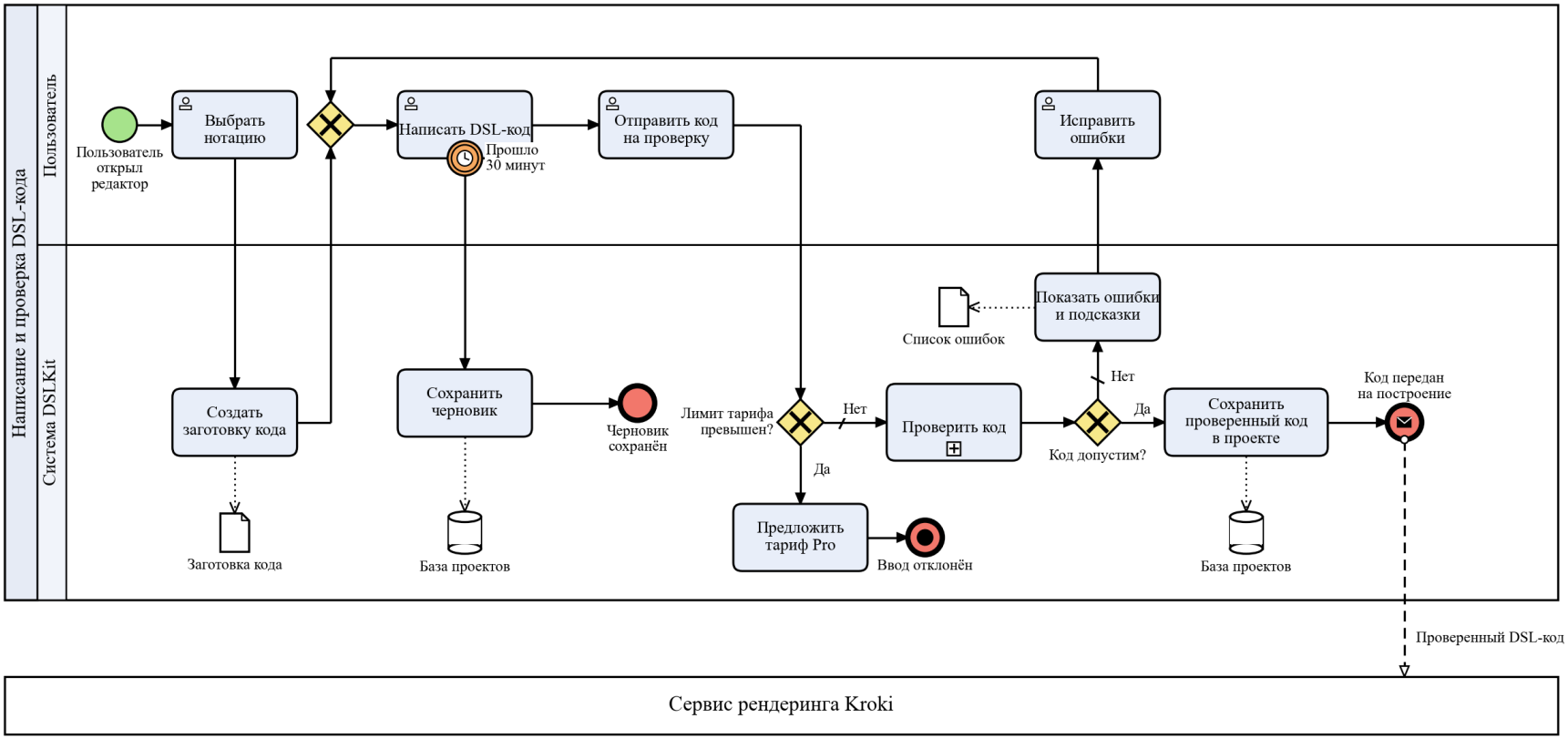


Рисунок 1.1 – Диаграмма процесса «Написание и проверка DSL-кода» в нотации BPMN

Основной путь проходит через проверку кода, а отклонения по лимиту тарифа и по бездействию пользователя завершаются отдельными конечными событиями.

1.4 Условия и альтернативы

Третий шаг: добавлены условия и альтернативные потоки. После отправки кода шлюз XOR «Лимит тарифа превышен?» сравнивает ввод с квотой тарифа. Ветка «Да» ведёт к задаче «Предложить тариф Pro» и к завершению «Ввод отклонён», ветка «Нет» передаёт код на проверку и отмечена знаком потока по умолчанию. После проверки шлюз «Код допустим?» направляет поток по ветке «Да» к сохранению проверенного кода или по ветке «Нет» к показу ошибок. Выходы обоих шлюзов подписаны. Ветка с ошибками возвращает пользователя к написанию кода через явный шлюз-слияние XOR, поэтому у задачи «Написать DSL-код» один входящий поток.

1.5 Неуспешные завершения

Четвёртый шаг: добавлены неуспешные завершения. Процесс заканчивается тремя способами: успешно событием «Код передан на построение» и неуспешно событиями «Ввод отклонён» (конечное событие типа «останов») и «Черновик сохранён». Последнее событие связано с граничным событием-таймером «Прошло 30 минут», которое прерывает задачу «Написать DSL-код» и сохраняет черновик, если пользователь долго не вводит код. Разные конечные события позволяют отличить успешный результат от неуспешных.

1.6 Артефакты

Пятый шаг: добавлены артефакты. Заготовка кода и список ошибок показаны объектами данных, база проектов показана хранилищем данных. Связь задачи с объектом обозначена ассоциацией (пунктирной линией). Задача «Создать заготовку кода» создаёт заготовку, задача «Показать ошибки и подсказки» создаёт список ошибок, а задачи «Сохранить проверенный код в проекте» и «Сохранить черновик» изменяют базу проектов.

1.7 Подпроцесс

Шестой шаг: раскрыт свёрнутый подпроцесс. Задача «Проверить код» обозначена на основной диаграмме знаком «плюс» и подробно показана на отдельной диаграмме, которая приведена на рисунке 1.2. Параллельный шлюз AND запускает две проверки одновременно: синтаксиса и правил нотации. Второй шлюз AND ждёт завершения обеих веток, после чего шлюз XOR «Ошибки найдены?» выбирает исход. При наличии ошибок формиру-

ется список ошибок и подпроцесс завершается событием «Код отклонён», иначе событием «Код принят». Поток управления не выходит за пределы подпроцесса.

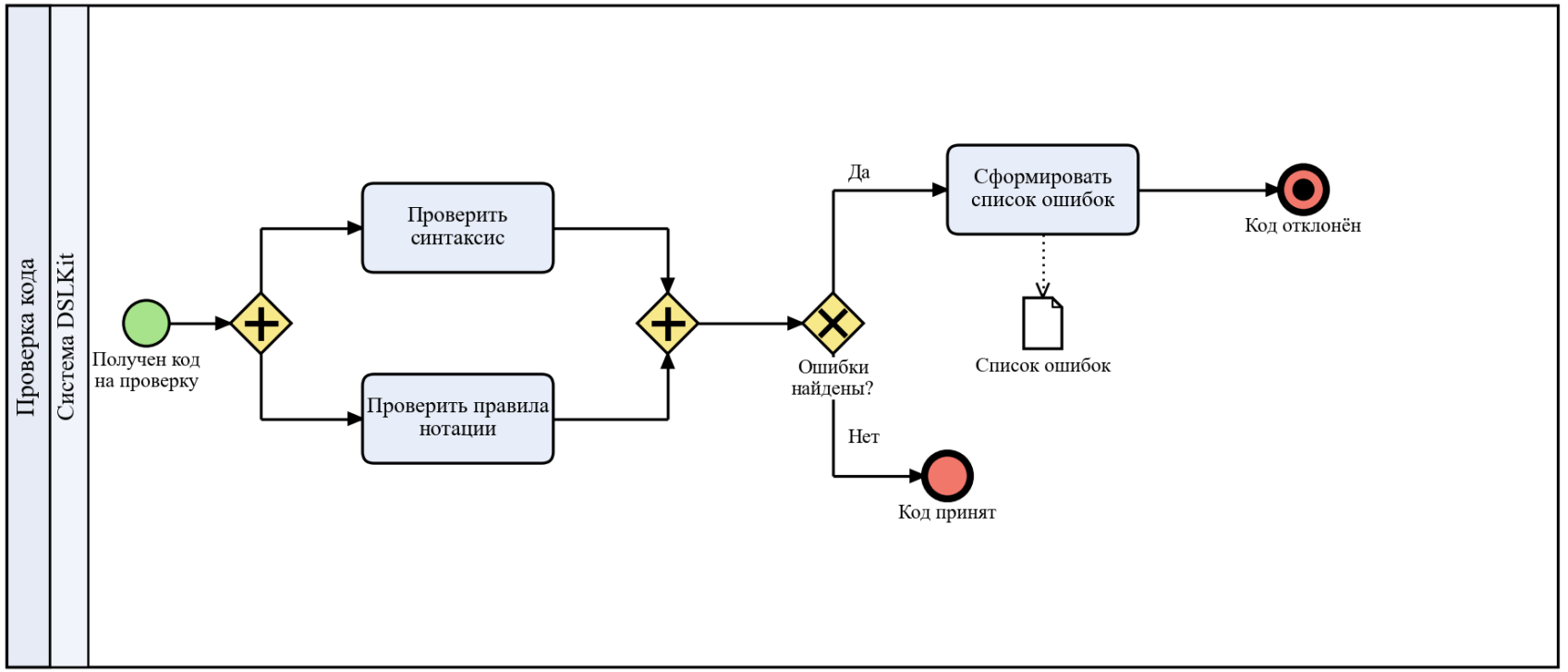


Рисунок 1.2 – Диаграмма подпроцесса «Проверить код»

Подпроцесс начинается получением кода и всегда заканчивается одним из двух исходов: код принят или код отклонён.

1.8 Потоки сообщений

Седьмой шаг: добавлены потоки сообщений к внешним пулам. Результат процесса передаётся в пул «Сервис рендеринга Kroki» сообщением «Проверенный DSL-код». Поток сообщений изображён пунктирной линией и присоединён к конечному событию, а не к шлюзу. Взаимодействие пользователя и системы показано потоками управления, так как обе дорожки находятся в одном пуле.

1.9 Соответствие правилам методички

Методичка рекомендует для диаграмм, которые читают люди, ограничивать алфавит нотации. Соответствие модели этим рекомендациям приведено в таблице 1.1. Выполнение формальных правил проверено программно по модели диаграммы: у стартового события нет входящих потоков, у конечных нет исходящих, у каждого шлюза-ветвления подписаны все выходы, потоки сообщений не присоединены к шлюзам.

Таблица 1.1 – Соответствие модели правилам методички

Правило	**Как выполнено**
Только пользовательские и ручные задачи	Действия человека показаны пользовательскими задачами с маркером. Шаги, которые система выполняет автоматически, показаны задачами без маркера типа, потому что подходящего типа среди допустимых нет
Только свёрнутые подпроцессы	Подпроцесс «Проверить код» свёрнут и раскрыт на отдельной диаграмме
Только шлюзы XOR и AND	Использованы XOR (три шлюза) и AND (пара в подпроцессе)
События: простое, таймер, сообщение, останов	Простое (старт, «Черновик сохранён»), таймер, сообщение (конец), останов («Ввод отклонён», «Код отклонён»)
Внешние участники свёрнутыми пулами	Сервис рендеринга Kroki
Закрытые пулы названы ролями, открытые процессами	Пул «Написание и проверка DSL-кода» назван по процессу
Дорожки названы по ролям	«Пользователь», «Система DSLKit»
Задачи в стиле «глагол плюс существительное»	«Выбрать нотацию», «Написать DSL-код», «Сохранить черновик» и другие
События названы свершившимся фактом	«Пользователь открыл редактор», «Прошло 30 минут», «Черновик сохранён»
Выходы XOR подписаны, поток по умолчанию отмечен	Подписи «Да» и «Нет», знак потока по умолчанию на ветке «Нет»
Успех и неуспех показаны разными конечными событиями	Три разных конечных события
Поток управления не выходит за пределы подпроцесса	Выполнено
Поток сообщений не присоединяется к шлюзам	Выполнено

Модель соответствует рекомендациям методички для диаграмм, которые читают люди, и не использует лишних элементов нотации.

1.10 Элементы модели

Элементы BPMN, использованные в модели, перечислены в таблице 1.2.

Таблица 1.2 – Использованные элементы нотации BPMN

Элемент	**Где использован**	**Назначение**
Открытый пул с дорожками	«Написание и проверка DSL-кода», дорожки «Пользователь» и «Система DSLKit»	Зоны ответственности ролей
Свёрнутый пул	Сервис рендеринга Kroki	Внешний участник без показа действий
Стартовое событие	Пользователь открыл редактор	Начало процесса
Граничное событие-таймер	Прошло 30 минут	Прерывание ожидания ввода кода
Конечные события	Код передан на построение, Ввод отклонён, Черновик сохранён	Успешное и неуспешные завершения
Пользовательская задача	Выбрать нотацию, Написать DSL-код, Отправить код на проверку, Исправить ошибки	Действия пользователя
Задача системы	Создать заготовку кода, Сохранить черновик, Предложить тариф Pro и другие	Автоматические шаги системы
Свёрнутый подпроцесс	Проверить код	Декомпозиция на отдельной диаграмме
Шлюзы XOR и AND	Слияние, Лимит тарифа, Код допустим, параллельные проверки	Выбор альтернативы и параллельное выполнение
Объект данных и хранилище	Заготовка кода, Список ошибок, База проектов	Документы и базы, сопровождающие процесс
Поток управления, поток сообщений, ассоциация	Вся модель	Порядок действий, обмен между пулами, связь с данными

Перечисленные элементы образуют набор, достаточный для описания

процесса, поэтому другие элементы нотации не потребовались.

ВЫВОДЫ

В ходе работы изучена нотация BPMN и построена модель процесса «Написание и проверка DSL-кода» сервиса DSLKit. Модель состоит из основной диаграммы с открытым пулом, двумя дорожками по ролям, внешним свёрнутым пулом, событиями, шлюзами, артефактами и потоком сообщений и из диаграммы свёрнутого подпроцесса «Проверить код». Построение выполнено по семи шагам алгоритма, алфавит нотации ограничен по рекомендациям методички, формальные правила проверены программно. Цель работы достигнута: освоено моделирование процесса в нотации BPMN, определены границы процесса, участники и результат.